Partitioning Methods

Designing an effective partitioning scheme can be challenging and requires careful consideration of the application requirements and the characteristics of the data being processed. Below are three of the most popular schemes used by various large-scale applications.

a. Horizontal Partitioning

Also known as sharding, horizontal data partitioning involves dividing a database table into multiple partitions or shards, with each partition containing a subset of rows. Each shard is typically assigned to a different database server, which allows for parallel processing and faster query execution times.

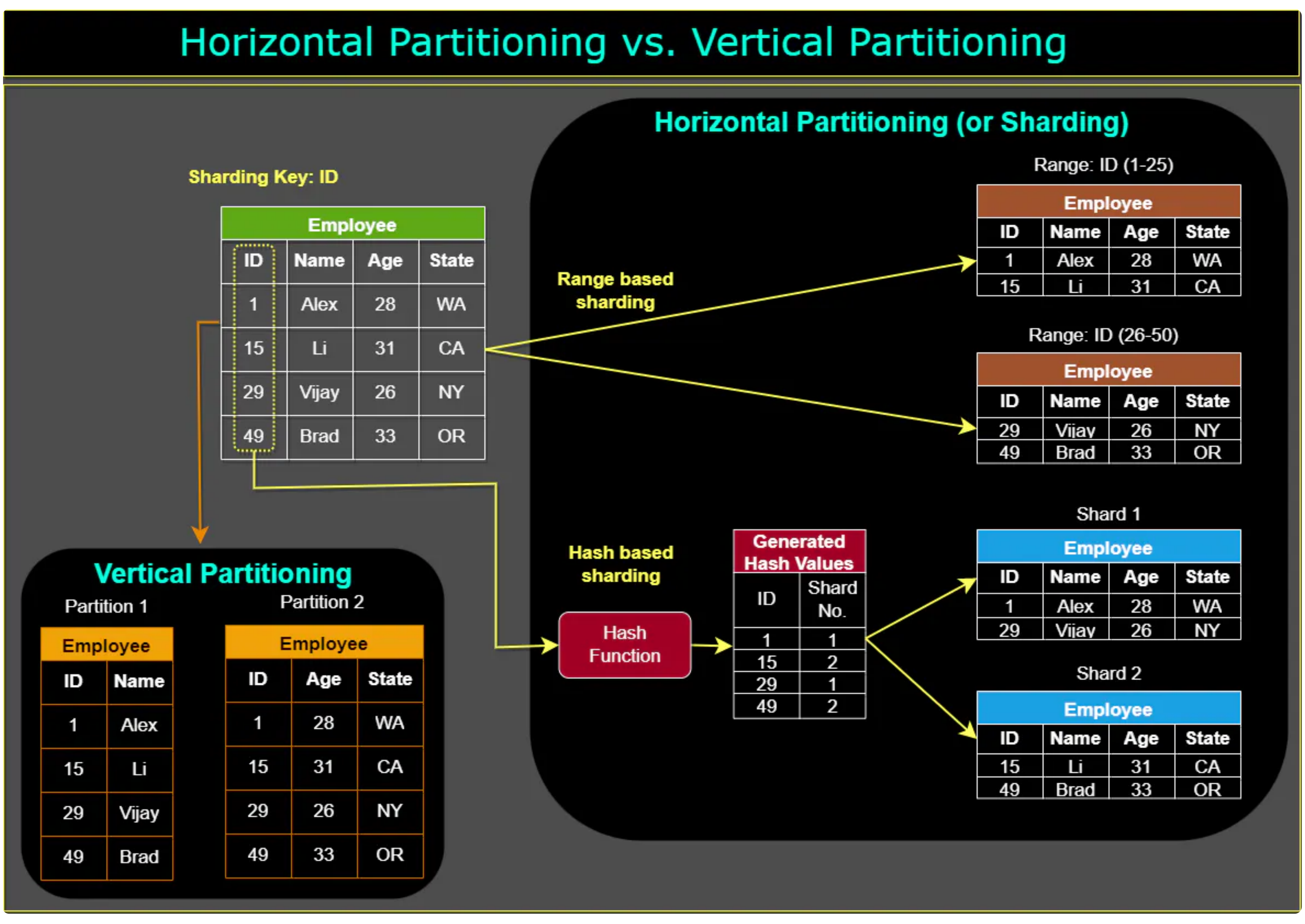
For example, consider a social media platform that stores user data in a database table. The platform might partition the user table horizontally based on the geographic location of the users, so that users in the United States are stored in one shard, users in Europe are stored in another shard, and so on. This way, when a user logs in and their data needs to be accessed, the query can be directed to the appropriate shard, minimizing the amount of data that needs to be scanned.

The key problem with this approach is that if the value whose range is used for partitioning isn't chosen carefully, then the partitioning scheme will lead to unbalanced servers. For instance, partitioning users based on their geographic location assumes an even distribution of users across different regions, which may not be valid due to the presence of densely or sparsely populated areas.

b. Vertical Partitioning

Vertical data partitioning involves splitting a database table into multiple partitions or shards, with each partition containing a subset of columns. This technique can help optimize performance by reducing the amount of data that needs to be scanned, especially when certain columns are accessed more frequently than others.

For example, consider an e-commerce website that stores customer data in a database table. The website might partition the customer table vertically based on the type of data, so that personal information such as name and address are stored in one shard, while order history and payment information are stored in another shard. This way, when a customer logs in and their order history needs to be accessed, the query can be directed to the appropriate shard, minimizing the amount of data that needs to be scanned.


Horizontal Partitioning vs. Vertical Partitioning
Sharding Key: ID
Employee
ID Name Age State
1 Alex 28 WA
15 Li 31 CA
29 Vijay 26 NY
49 Brad 33 OR
Horizontal Partitioning (or Sharding)
Range based sharding
Range: ID (1-25)
Employee
ID Name Age State
1 Alex 28 WA
15 Li 31 CA
Range: ID (26-50)
Employee
ID Name Age State
29 Vijay 26 NY
49 Brad 33 OR
Hash based sharding
Hash Function
Generated Hash Values
ID Shard No.
1 1
15 2
29 1
49 2
Shard 1
Employee
ID Name Age State
1 Alex 28 WA
29 Vijay 26 NY
Shard 2
Employee
ID Name Age State
15 Li 31 CA
49 Brad 33 OR
Vertical Partitioning
Partition 1
Employee
ID Name
1 Alex
15 Li
29 Vijay
49 Brad
Partition 2
Employee
ID Age State
1 28 WA
15 31 CA
29 26 NY
49 33 OR

c. Hybrid Partitioning

Hybrid data partitioning combines both horizontal and vertical partitioning techniques to partition data into multiple shards. This technique can help optimize performance by distributing the data evenly across multiple servers, while also minimizing the amount of data that needs to be scanned.

For example, consider a large e-commerce website that stores customer data in a database table. The website might partition the customer table horizontally based on the geographic location of the customers, and then partition each shard vertically based on the type of data. This way, when a customer logs in and their data needs to be accessed, the query can be directed to the appropriate shard, minimizing the amount of data that needs to be scanned. Additionally, each shard can be stored on a different database server, allowing for parallel processing and faster query execution times.

← Previous
Introduction To Data Partitioning

Next →
Data Sharding Techniques

☐ Mark as Completed